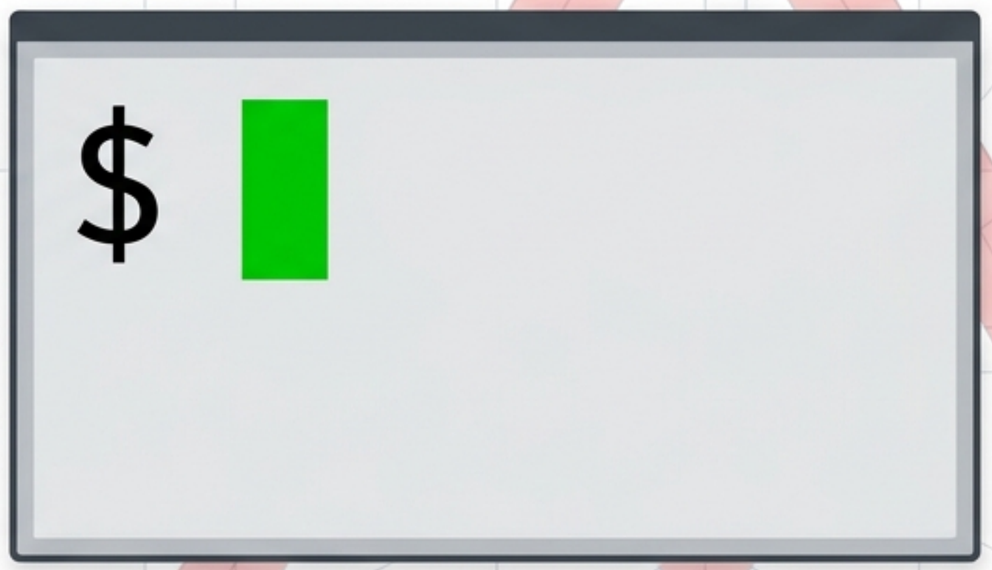
Triage and Resolution: RDS Connection Failures

NorthBank Day-2 Operations Playbook | Lab 09



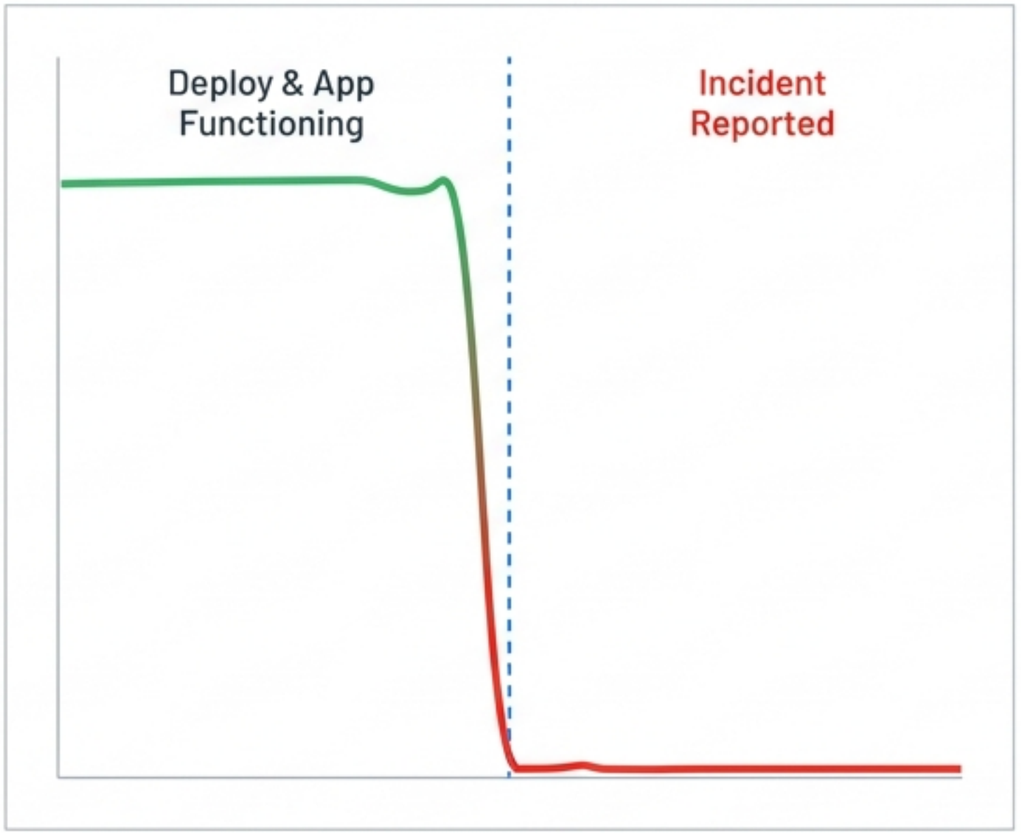
NotebookLM

[SEV-1]

Reporter: Marco (App Team)

Status: INVESTIGATING

“Transfers are failing — the app can’t connect to the database.”



NotebookLM

The Command Resolution Pipeline

1. Deploy Foundation

```
$ terraform apply
-var="create_datab
ase=true"
```

Takes ~3-5 min to spin up environment

2. Extract Variables

```
$ export DB_SG,
APP_SG, DB_ID
```

Captures security groups and instance IDs

3. Trigger the Incident

```
$ ./break.sh
```

Malicious script silently removes the app→DB port-3306 rule

The environment is live. The database is isolated. The SEV-1 investigation begins now.

NotebookLM

Initial Triage: The Trap of 'Available'

```
$ describe-db-instances
{
  DBInstances: [
    {
      DBInstanceIdentifier: 'northbank-prod-db',
      DBInstanceStatus: 'available'
```

The engine is powered on and healthy.

```
$ describe-events
{
  Events: []
}

No recent events found.
```

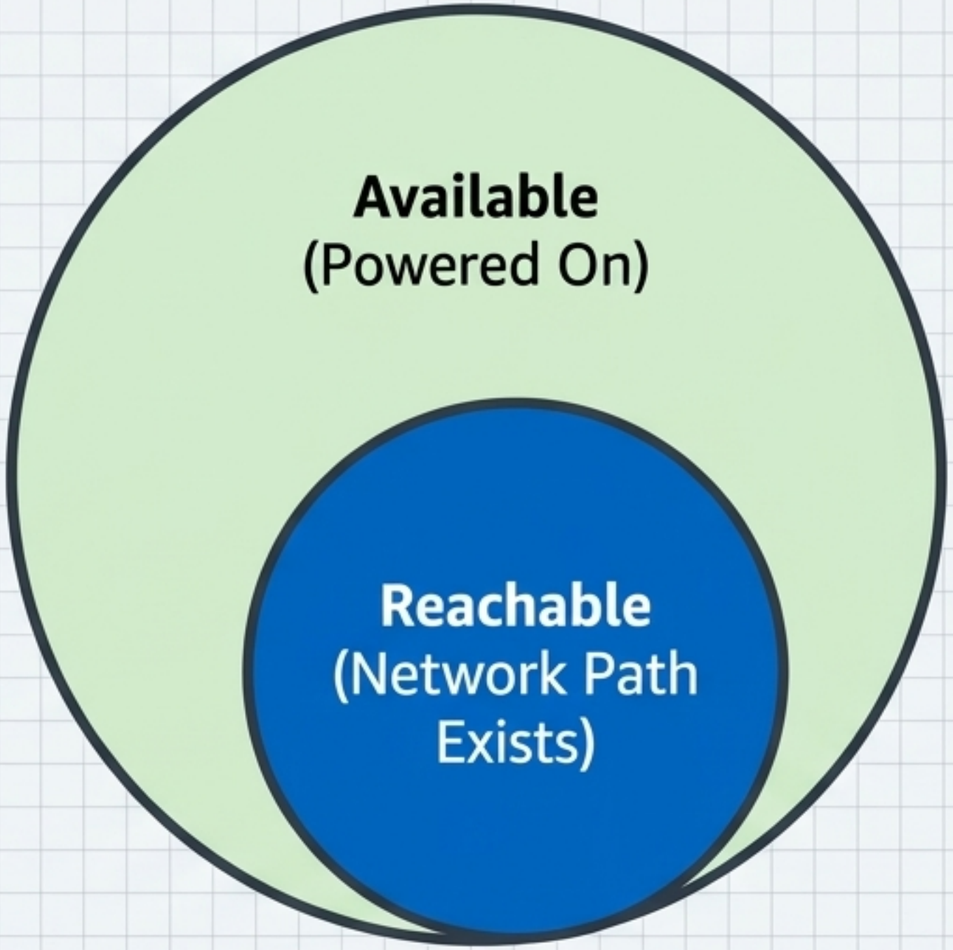
The RDS control plane reports zero internal errors.

The Paradox: The DB says everything is fine, but the App says transfers are failing.

NotebookLM

The Illusion of Availability

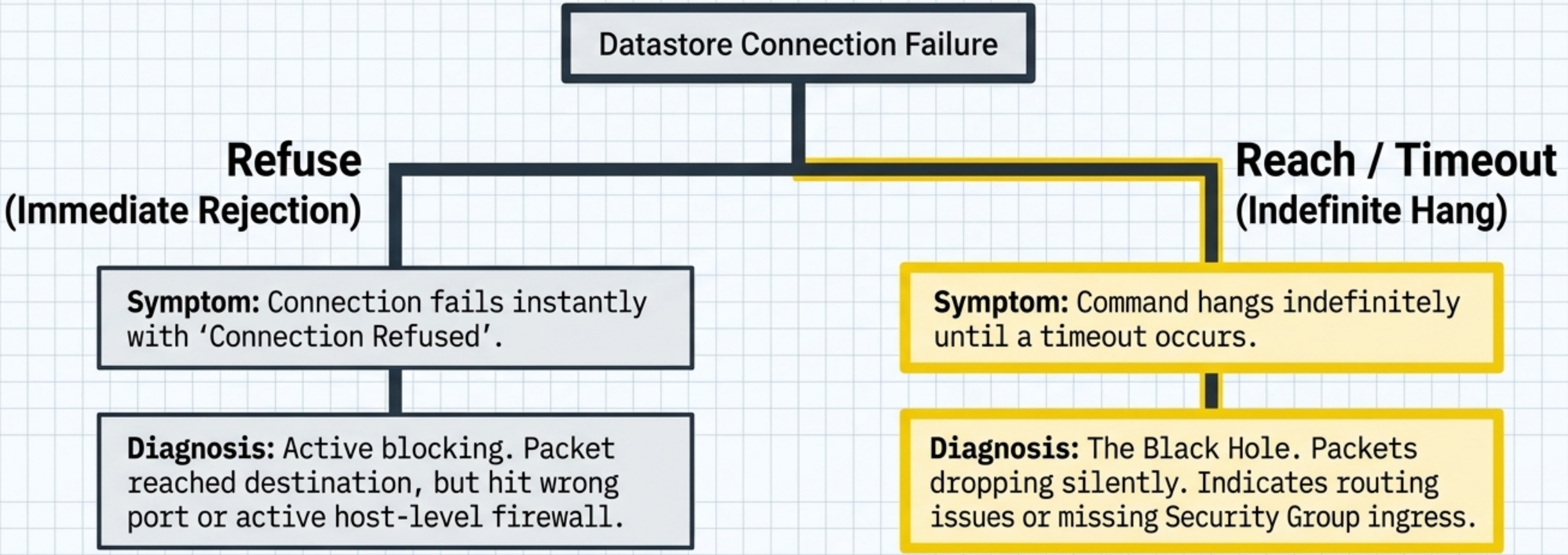
AWS “Available”	Actually “Reachable”
The EC2 instance is powered on. The database engine service is active.	A verified network path exists. Target ports are actively listening and unblocked.
`describe-db-instances`	Network configurations & Security Groups.
Internal to the machine.	External to the machine (The Network).



NotebookLM

The Diagnostic Fork

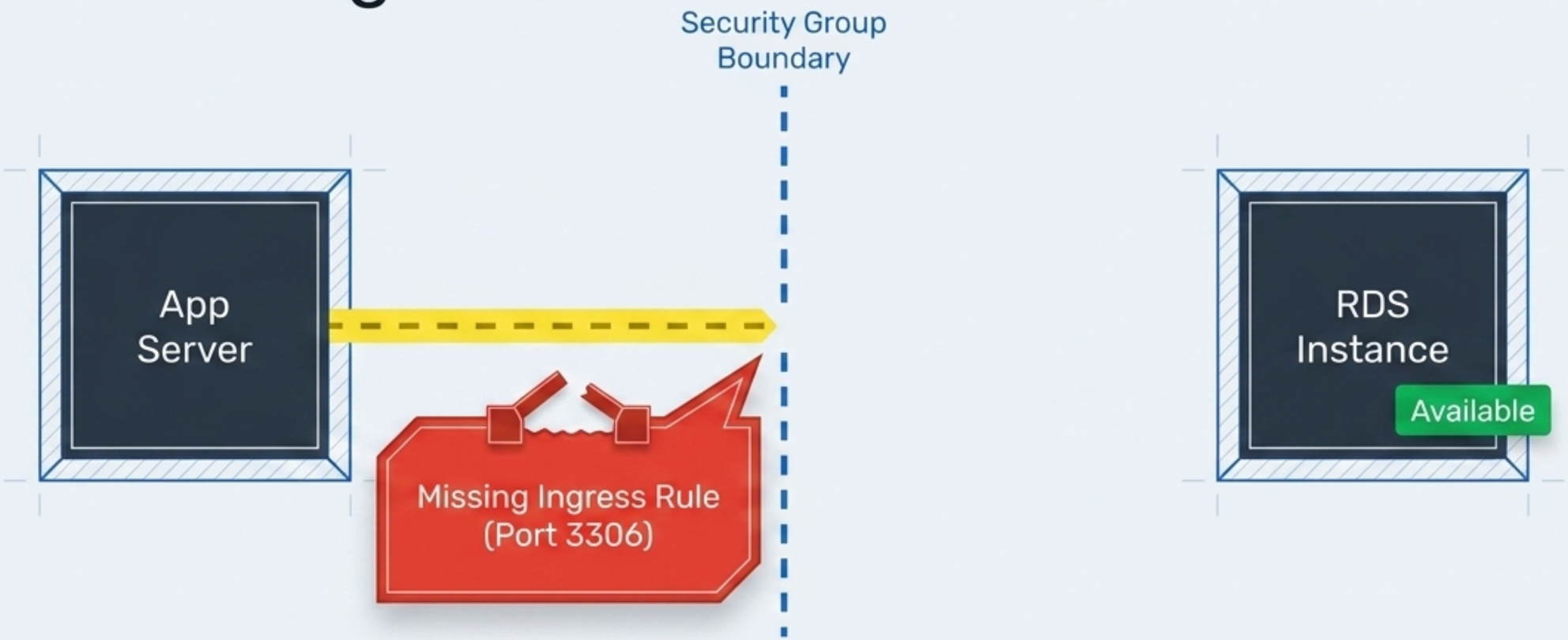
Determine the physical reality of the network failure before typing commands.



Marco's ticket states the app "can't connect" and transfers are timing out. We are dealing with a Reach/Timeout scenario.

NotebookLM

Architectural Reality: The Drawbridge



The application is sending traffic to a black hole. The database is waiting for traffic that never arrives. The boundary is broken.

NotebookLM

Confirming the Root Cause

```
$ describe-security-groups --group-ids $DB_SG
{
   SecurityGroups: [
      {
         GroupName: 'northbank-db-sg',
         GroupId: 'sg-0a1b2c3d4e5f',
         IpPermissions: [],
         VpcId: 'vpc-987654321'
      }
   ]
}
```



Diagnostic Conclusion: The break.sh script successfully wiped the ingress rules. Port 3306 is completely closed to the application. The root cause is confirmed.

NotebookLM

The Treatment

```
$ authorize-security-group-ingress --group-id $DB_SG
  --protocol tcp --port 3306 --source-group $APP_SG
```

Precision scoping: Opening only the exact port, only to the exact source.

Connection Restored

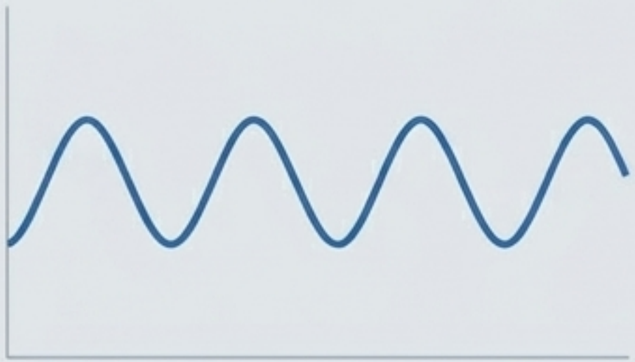


NotebookLM

The Day-2 Monitoring Baseline

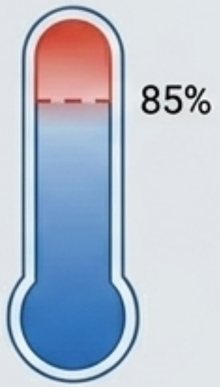
The “Holy Trinity” of RDS metrics to monitor alongside network path integrity.

DatabaseConnections



Why it matters: Tracks active traffic. A sudden drop to zero indicates a network/security group failure.

FreeStorageSpace



Why it matters: The silent killer. If an instance runs out of storage, it will lock up entirely, regardless of network health.

CPUUtilization



Why it matters: Indicates query efficiency and sheer load. High CPU with low connections points to unoptimized queries.

Synthesis: Monitor these three metrics to catch 90% of datastore SEV-1s before they happen.

NotebookLM

SEV-1 Resolved. Connection Restored.

“Status is a state of the machine. Reachability is a state of the network.”

The Post-Mortem Checklist

1

Never trust “Available”.
Always verify the network path.

2

Use the Fork.
Did it refuse (active block) or timeout (black hole)?

3

Check the Boundary.
Security Groups fail silently; check them before checking the DB engine.

NotebookLM